

Module 11 - Firebase Data Model, Rules & Trusted Logic

Firestore schema, indexes, access control, Cloud Functions boundaries

SOURCE OF TRUTH: This resource intentionally uses plain JavaScript. The locked stack is React + JavaScript + Firebase + Material UI (MUI). Do not introduce Next.js, TypeScript, shadcn/ui, Redux, or another backend framework unless the project owner explicitly changes this specification.

Scope

This module is the backend contract. The React app is not trusted. Firebase Security Rules are mandatory for group-scoped authorization, while Cloud Functions handle operations requiring trusted computation or controlled fan-out.

Top-level collections

```
users/{uid}
groups/{groupId}
inviteLinks/{inviteId}
(optional) systemConfig/{docId}
```

Subcollections

```
groups/{groupId}/members/{uid}
groups/{groupId}/plans/{planId}
groups/{groupId}/bookings/{bookingId}
groups/{groupId}/expenses/{expenseId}
groups/{groupId}/polls/{pollId}
groups/{groupId}/activities/{activityId}
... nested members / participants / splits / itinerary / votes
```

Security principle

Client-side route guards are UX only. Real protection must be expressed in Firestore Security Rules and trusted backend functions. A malicious user must not gain access by editing browser JavaScript or request payloads.

Rule helpers

- `isSignedIn()`
- `isGroupMember(groupId)`
- `isGroupAdmin(groupId)`
- `isGroupOwner(groupId)`
- `isSelf(uid)`
- `isPlanParticipant(groupId, planId)`

Rules matrix

Data	Read	Create	Update	Delete
users/self	self	self	self	self or admin policy
groups	member	signed-in	owner/admin	owner
members	members	owner/admin	owner/admin	owner/admin or self-leave
plans	members	members	organizer/admin	organizer/admin

Data	Read	Create	Update	Delete
expenses	members	members	creator/admin	creator/admin
activities	members	trusted writes preferred	No direct	No direct
invites	admin	admin	admin	admin

Cloud Functions candidates

- Trusted settlement calculation if later needed server-side
- Invite validation for high-security tokens
- Notification fan-out
- Cleanup/archival jobs
- Any operation requiring privileged cross-document writes

Indexes

Create Firestore composite indexes only when query patterns require them. Typical candidates: groupId + createdAt, groupId + status + startAt, paidBy + createdAt, memberId + plan status (depending on final denormalization). Do not create indexes blindly.

Acceptance criteria

- Rules tested with authorized and unauthorized cases.
- A user cannot read a group they are not a member of.
- A member cannot perform owner-only changes.
- Sensitive writes are not trusted merely because the UI hides a button.